

Phase 01 — Internet Fundamentals (Complete Notes)

How the Internet works, IP addresses, MAC, routers, switches, NAT, ports, TCP/UDP, OSI model. (Lesson 01 — How the Internet Works — has its own detailed file. This chapter covers everything else.)

1. Quick recap of Lesson 01

- The Internet = network of networks, connected by cables + radio, speaking shared **protocols**.
- **Client** asks, **server** answers. They are roles, not machines. Servers never initiate.
- Data travels as **packets** (~1,500 bytes), each with source address, destination address, and sequence number. Routers forward packets hop by hop.

Open question from Lesson 01: *what exactly is an "address"?* → This chapter.

2. IP Addresses

What is it?

An IP (Internet Protocol) address is the unique number identifying a machine on a network, e.g. 142.250.190.78. Packets carry a destination IP so routers know where to send them.

Analogy: a postal address for a computer. No address → the postman (router) cannot deliver.

IPv4

- Format: 4 numbers, 0-255 each, separated by dots: 192.168.0.101
- Each number is 8 bits → total 32 bits → about **4.3 billion** possible addresses.
- Problem: the world has far more devices than 4.3 billion. IPv4 ran out. Two fixes were invented: **NAT** (below) and **IPv6**.

IPv6

- 128 bits, written in hex: 2404:6800:4002:81e::200e
- Enough addresses for every grain of sand on Earth. Adoption is gradual; most systems today run both (dual stack).

Public IP vs Private IP

- **Public IP** — globally unique, reachable from the Internet. Your router gets one from your ISP. A production server (EC2, VPS) has one.
- **Private IP** — only valid inside a local network. Reserved ranges (memorize these):

10.0.0.0	– 10.255.255.255	(10.x.x.x)	← AWS VPCs commonly use this
172.16.0.0	– 172.31.255.255		← Docker commonly uses this
192.168.0.0	– 192.168.255.255		← home routers commonly use this

Every device in your home has a private IP (e.g. `192.168.0.101`), but they all share ONE public IP when talking to the Internet.

Analogy: private IP = apartment number (Flat 4B — meaningless outside the building). Public IP = the building's street address.

Localhost — 127.0.0.1

- `127.0.0.1` (name: `localhost`) always means **this machine itself**. Traffic to it never leaves your computer — no cable, no router.
- That's why `http://localhost:8080` works with WiFi off, and why nobody else can open *your* localhost.
- `0.0.0.0` when used by a *server* means "listen on ALL my network interfaces" — accept connections from localhost AND from the network. Important later in Docker and NGINX configs.

Commands

```
ip addr          # your machine's IPs on each interface (look for "inet")
curl ifconfig.me # your PUBLIC IP (as the Internet sees you)
ping 127.0.0.1   # talk to yourself – works even offline
```

Compare `ip addr` output with `curl ifconfig.me` — they differ. That difference is NAT (below).

3. Network Interfaces and MAC Addresses

Network interface

The hardware/software port through which a machine connects to a network. Typical names on Linux:

```
lo          → loopback (localhost lives here)
eth0/enp... → wired Ethernet
wlan0/wlp... → WiFi
docker0     → virtual interface created by Docker (Phase 6!)
```

One machine can have many interfaces, each with its own IP. `ip addr` lists them.

MAC address

- A permanent hardware serial number burned into every network card: `a4:5e:60:d3:8c:1f` (48 bits).
- **IP vs MAC:** IP = your postal address (changes when you move networks). MAC = your national ID (permanent, identifies the physical card).
- MAC is used for delivery within the *local* network only (switches use it). It never survives past your router — the Internet routes purely on IPs.

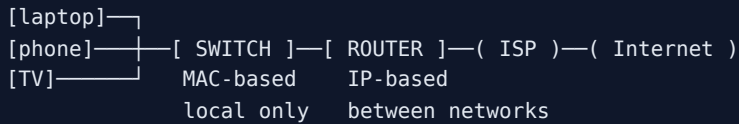
4. Routers and Switches

Switch

- Connects devices **within one local network**. Delivers frames using **MAC addresses**.
- Analogy: the reception desk inside one office building — knows every employee by face, delivers internal mail. Knows nothing about other buildings.

Router

- Connects **different networks** to each other. Forwards packets using **IP addresses**.
- Maintains a *routing table*: "for destinations like X, next hop is Y." No router knows the whole path — each just knows the best next step.
- Analogy: the inter-city postal sorting hub.



Your home "router" box is really router + switch + WiFi access point + DHCP server + NAT in one.

5. NAT — Network Address Translation

What problem does it solve?

Not enough public IPv4 addresses for every device. NAT lets a whole private network share ONE public IP.

How it works

Your router rewrites packets on the way out and remembers who asked:

```
laptop 192.168.0.101:53000 → google.com:443
router rewrites source to: 103.120.5.9:61001 ← public IP
router notes: "61001 belongs to 192.168.0.101:53000"

reply from google → 103.120.5.9:61001
router looks up 61001 → forwards to laptop 192.168.0.101
```

Analogy: an office receptionist. All employees call out via one office phone number; the receptionist remembers who dialed whom and routes callbacks to the right desk.

Consequence you will hit in real work

Outside machines **cannot initiate** connections to a device behind NAT (the receptionist doesn't know whom the call is for). This is why you can't just run a server on your laptop and have the world reach it — and why we deploy on VPS/cloud machines that have real public IPs (Phase 7). It also acts as accidental security.

6. Ports

What problem do they solve?

An IP finds the *machine* — but which *program* on it? A machine runs many network programs at once. **Ports** (numbers 1–65535) identify which program a packet is for.

Analogy: IP = building address, port = apartment number. `142.250.4.100:443` = building 142.250.4.100, apartment 443.

Standard ports (memorize)

22	SSH	80	HTTP	443	HTTPS
25	SMTP (mail)	53	DNS	3306	MySQL
5432	PostgreSQL	6379	Redis	8080	common dev HTTP (Spring Boot default)

- Ports below 1024 are "privileged" (need root to listen on).
- A server program "listens" on a port. Only ONE program can listen on a given port at a time → the classic error `Address already in use: bind` when you start Spring Boot twice.

Commands

```
sudo ss -tulpn          # what's listening on which port (modern)
sudo lsof -i :8080      # which process owns port 8080
kill <PID>              # stop it
```

7. TCP and UDP

Two protocols for actually delivering packets. Both use IPs and ports; they differ in guarantees.

TCP — Transmission Control Protocol

- **Connection-based**: starts with the 3-way handshake:

```
client — SYN —> server    "want to talk?"
client ← SYN-ACK — server  "yes, ready"
client — ACK —> server     "great, starting" → connection established
```

- Guarantees: **all packets arrive, in order, uncorrupted**. Lost packets are re-sent automatically. Receiver acknowledges (ACK) everything.
- Cost: slower, more overhead.
- Used by: HTTP/HTTPS, databases, SSH, email — anything where a missing byte is unacceptable. **Everything in our course rides on TCP.**
- Analogy: registered mail with delivery confirmation and automatic resend.

UDP — User Datagram Protocol

- No handshake, no ACK, no retransmit, no ordering. Fire and forget.
- Cost of TCP's guarantees removed → very fast, low latency.
- Used by: video calls, live streams, online games, DNS queries. (A lost video frame is better skipped than re-sent late.)
- Analogy: throwing postcards into the mailbox — most arrive, you never check.

Rule of thumb

Correctness required → TCP. Speed matters and losses are tolerable → UDP.

8. The OSI Model — the map of network layers

A conceptual model splitting networking into 7 layers. Each layer only talks to the layer above/below. Interviews love it; engineers use layers 3, 4, 7 daily.

7	Application	HTTP, DNS, SMTP	← your Spring Boot lives here
6	Presentation	TLS encryption, encoding	
5	Session	connection sessions	
4	Transport	TCP / UDP, ports	← "Layer 4 load balancer" (AWS NLB)
3	Network	IP, routers	← packets routed here
2	Data Link	MAC, switches, Ethernet	
1	Physical	cables, radio, light	

Mnemonic (bottom-up): **P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way.

Why you actually care

- Production vocabulary: "L4 vs L7 load balancer" (Phase 10), "L3 firewall rule".
- Debugging ladder — check from the bottom up:
 1. Cable/WiFi up? (L1-2)
 2. ping <ip> works? (L3)
 3. Port open — nc -zv host 443 ? (L4)
 4. App answers — curl ? (L7)

The simpler **TCP/IP model** (what the Internet actually uses) collapses these into 4: Link → Internet → Transport → Application.

9. The full picture — one request with everything labeled

```
Browser wants  https://api.example.com/users

[L7] browser builds HTTP request
[L4] TCP: connect to port 443, 3-way handshake, split into segments
[L3] IP: each packet stamped  src=192.168.0.101  dst=<server public IP>
[L2] MAC: frame addressed to home router's MAC
[L1] bits over WiFi radio waves
    |
    ▼
Home router: NAT rewrites src → public IP :random-port, remembers mapping
    ▼
10–25 Internet routers, each forwarding by destination IP
    ▼
Server machine: has public IP, program listening on port 443
[L4] TCP reassembles segments in order
[L7] Spring Boot receives the HTTP request, responds
    ▼
Response packets flow back; NAT maps them to the laptop; browser renders
```

If you can narrate this diagram from memory, Phase 1 is yours.

10. Common mistakes

- Confusing IP with MAC (postal address vs national ID; L3 vs L2).

- Thinking `localhost` is reachable by teammates on your WiFi — it never leaves your machine. Use your private IP (`192.168.x.x`) for that.
- Binding a server to `127.0.0.1` and wondering why the network can't reach it (should be `0.0.0.0`). You WILL hit this with Docker.
- Thinking NAT is a firewall by design — it blocks inbound as a side effect, but it is not a security feature.
- Saying "TCP is better than UDP" — they solve different problems.

11. Best practices

- Learn to instinctively run the debugging ladder: `ping` → `ss /port check` → `curl` .
- Always know: what IP am I on, what's my public IP, what ports are my apps using.
- Prefer standard ports in production (80/443 outside, anything behind the proxy inside).

12. Interview questions

1. IPv4 vs IPv6 — why does IPv6 exist?
2. Public vs private IP? Which ranges are private?
3. What is NAT and why can't an outside machine connect to your laptop at home?
4. What is 127.0.0.1? Difference between binding to 127.0.0.1 vs 0.0.0.0?
5. Router vs switch — which OSI layer, which address type?
6. Explain the TCP 3-way handshake. Why does HTTP use TCP, but video calls use UDP?
7. Two programs want port 8080 — what happens? How do you find/fix it?
8. Name the OSI layers and one protocol per layer. What does "L7 load balancer" mean?

13. Lab

```
# identity
ip addr                # find your interfaces + private IP
curl ifconfig.me       # public IP → different! that's NAT in action
ip route               # find your "default gateway" = your router's IP

# ports & servers
python3 -m http.server 8000      # terminal 1
sudo ss -tulpn | grep 8000       # terminal 2: see it LISTENing
python3 -m http.server 8000      # terminal 3: watch it FAIL — port taken

# reach your own machine from another device
# open http://<your-private-ip>:8000 from your PHONE on the same WiFi
# → works via private IP, would NOT work via localhost. Understand why.

# TCP handshake in real life
curl -v https://example.com      # "Connected to ..." line = handshake done
```

14. ASSIGNMENT 01 (submit answers to Claude for review)

1. Run `ip addr` and `curl ifconfig.me` . Paste both outputs and explain, in your own words, why the IPs differ and what NAT did.
2. Draw (ASCII, from memory) the journey of a request from your browser to a Spring Boot server, labeling: private IP, public IP, NAT, router, port, TCP.
3. Your Spring Boot app on your laptop must be shown to a teammate on the same office WiFi. Exactly what URL do you give them, and what conditions must be true for it to work?

4. Explain why `application.properties` setting `server.address=127.0.0.1` would break a Dockerized Spring Boot app. (Reason from first principles — we haven't done Docker yet, guess bravely.)
5. Answer interview questions 3, 4, 6 from memory, in writing.